

Joint Intent Classification and Slot Filling on ATIS

Kaniz Fatima (267475)

University of Trento

kaniz.fatima@studenti.unitn.it

1. Introduction

This report covers joint intent classification and slot filling on the ATIS corpus [1], a standard benchmark for the two tasks [2]. Each utterance carries one sentence-level intent and one IOB slot tag per word, so a single model has to produce both a sentence label and a token-level sequence. Part 2.A trains a decoder-only Transformer [3] from scratch, improving it one change at a time. Part 2.B fine-tunes pretrained GPT2 [4] and BERT [5] on the same two tasks, which additionally requires reconciling word-level labels with sub-word tokenization. Comparing the two isolates what pretraining contributes on a corpus of roughly 4,500 training utterances. Dataset statistics are in Table 1.

2. Implementation details

ATIS ships no development split, so 10% is held out of training, stratified by intent; four intents occur once and cannot be stratified, so they stay in training. Every configuration is trained with three seeds and reported as mean $\pm$ standard deviation, because on a corpus this small a single run varies enough to be mistaken for a real effect. Selection uses the development joint score $J = \frac{1}{2}F1_{slot} + \frac{1}{2}Acc_{intent}$; test is evaluated once, on the selected checkpoint. Optimisation uses AdamW. The loss is the sum of a token-level cross-entropy for slots and an utterance-level cross-entropy for intents. Slot F1 is computed with the provided `conll` evaluator, and gold labels are read from the original strings rather than through the label vocabulary, so a slot never seen in training is still scored correctly instead of silently counting as padding.

For Part 2.A a `cls` token is appended to every utterance and its final hidden state feeds the intent head. It has to go at the end rather than the start because attention is causal: only the last position has seen the whole utterance. The slot head classifies every other position, and dropout before both heads is the Step 2 change.

Part 2.B keeps the same two heads but changes where the sentence representation comes from: BERT uses `[CLS]`, while GPT2 uses the last non-padding token, for the same causal reason. Sub-tokenization is handled by assigning each word's label to its *first* sub-token and masking every continuation piece and special token with `-100`; at evaluation, predictions are read back only at those first-piece positions, recovering exactly one tag per original word.

Declaration of AI tool use. AI tools were used for code scaffolding and report organisation. All experiments were executed by the author, and every number reported here is the measured output of those runs.

3. Results

The Part 2.A learning-rate search (Table 2) selects 10^{-3} at 93.94 development joint score. The ranking is clear and the margin over 3×10^{-4} (89.74) is far larger than the seed spread,

so the rate is fixed before any architectural change.

The incremental search (Table 3, Figures 1 and 2a) improves the baseline from 93.94 to 96.17 development joint score. Widening the model and adding a second layer both help, and doubling the attention heads is the one ambiguous step: test slot F1 falls from 88.33 to 86.91 while intent accuracy rises, so the change is not uniformly beneficial. Dropout gives the largest single gain of the whole search, adding 0.92 development joint score and 1.78 test slot F1, and is the selected configuration at 90.39 test slot F1 and 92.83 intent accuracy.

Part 2.B is clearly stronger (Table 4, Figures 2b and 3). The best model, BERT at 5×10^{-5} , reaches 95.61 test slot F1 and 97.69 intent accuracy. The learning rate matters far less than the backbone: moving BERT from 3×10^{-5} to 5×10^{-5} changes slot F1 by 0.21, while switching from GPT2 to BERT changes it by 2.57.

That backbone gap is the most informative result. BERT beats GPT2 by 2.57 points on slot F1 but only 0.94 on intent accuracy. Slot filling is a per-token decision that benefits from seeing the words after the target, which a causal decoder structurally cannot do, whereas intent is a single sentence-level decision and GPT2's final token has already read the whole utterance. The asymmetry follows from the architectures rather than from tuning.

Against Part 2.A (Table 5), fine-tuning gains 5.22 slot F1, 4.86 intent accuracy and 5.04 joint score. Convergence is also much faster: 7.7–13.3 epochs against 28.7–36.0 for the scratch model, since the pretrained weights already encode most of what the task needs and training mainly has to attach the two heads.

4. References

- [1] C. T. Hemphill, J. J. Godfrey, and G. R. Doddington, "The ATIS spoken language systems pilot corpus," in *Proceedings of the DARPA Speech and Natural Language Workshop*, 1990, pp. 96–101. [Online]. Available: <https://aclanthology.org/H90-1021/>
- [2] H. Weld, X. Huang, S. Long, J. Poon, and S. C. Han, "A survey of joint intent detection and slot filling models in natural language understanding," *ACM Computing Surveys*, vol. 55, no. 8, pp. 156:1–156:38, 2022. [Online]. Available: <https://doi.org/10.1145/3547138>
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [4] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423/>

Tables and Figures

Table 1: *Dataset statistics. The development split was carved out of the original ATIS training data. Label counts are those observed in the training split.*

Split / resource	Count	Notes	Usage
Original ATIS training set	4,978	Before internal split	Source for train/dev
Training split	4,480	Intent-stratified	Parameter learning
Development split	498	10% of training data	Model selection
Test split	893	Original ATIS test set	Final evaluation
Intent labels	22	Seen in training	Intent output
Slot labels	119	IOB slot labels	Slot output
Part 2.A vocabulary	867	Includes pad, unk, cls	Scratch input

Table 2: *Part 2.A learning-rate search, architecture fixed at the baseline. Mean $\pm$ std over three seeds; bold marks the selected rate.*

Experiment	LR	Best ep.	Dev Slot F1	Dev Intent	Dev Joint	Test Slot F1	Test Intent
base_lr1e-3	0.0010	36.00	92.30 $\pm$ 0.76	95.58 $\pm$ 0.53	93.94 $\pm$ 0.58	86.60 $\pm$ 1.21	91.04 $\pm$ 1.34
base_lr5e-4	0.0005	38.00	89.44 $\pm$ 1.03	94.24 $\pm$ 1.01	91.84 $\pm$ 0.26	84.13 $\pm$ 1.86	89.36 $\pm$ 0.62
base_lr3e-4	0.0003	37.67	85.11 $\pm$ 0.86	94.38 $\pm$ 0.60	89.74 $\pm$ 0.14	80.40 $\pm$ 1.17	87.61 $\pm$ 0.06

Table 3: *Part 2.A scratch Transformer experiments. Each row adds one change and keeps it. Bold marks the configuration selected by development joint score.*

Experiment	d_{model}	Heads	Layers	FF	Drop	Dev joint	Test Slot F1	Test Intent	Params
step0_baseline	128	2	1	256	0.0	93.94 $\pm$ 0.58	86.60 $\pm$ 1.21	91.04 $\pm$ 1.34	0.27M
step1_width192	192	2	1	256	0.0	94.69 $\pm$ 0.37	88.33 $\pm$ 0.43	90.59 $\pm$ 1.66	0.45M
step2_heads4	192	4	1	256	0.0	94.61 $\pm$ 0.48	86.91 $\pm$ 0.46	91.30 $\pm$ 2.69	0.45M
step3_depth2	192	4	2	256	0.0	94.96 $\pm$ 1.00	88.12 $\pm$ 0.39	91.34 $\pm$ 0.72	0.70M
step4_ffn512	192	4	2	512	0.0	95.25 $\pm$ 0.46	88.61 $\pm$ 0.58	90.93 $\pm$ 1.36	0.90M
step5_dropout01	192	4	2	512	0.1	96.17 $\pm$ 0.28	90.39 $\pm$ 0.48	92.83 $\pm$ 0.85	0.90M

All rows use the selected learning rate 10^{-3} .

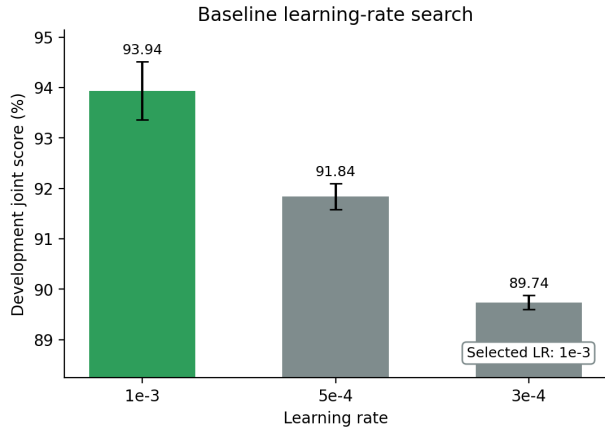
Table 4: *Part 2.B pretrained fine-tuning. Mean $\pm$ std over three seeds; bold marks the best model by development joint score.*

Experiment	Backbone	LR	Best ep.	Dev Joint	Test Slot F1	Test Intent	Params
gpt2_lr5e5	GPT-2	5e-05	13.33 $\pm$ 1.15	98.01 $\pm$ 0.05	93.04 $\pm$ 0.38	96.75 $\pm$ 0.22	124.55M
gpt2_lr3e5	GPT-2	3e-05	11.67 $\pm$ 2.52	97.83 $\pm$ 0.18	92.01 $\pm$ 0.97	96.16 $\pm$ 0.17	124.55M
bert_lr5e5	BERT	5e-05	13.33 $\pm$ 2.89	98.96 $\pm$ 0.10	95.61 $\pm$ 0.07	97.69 $\pm$ 0.13	109.59M
bert_lr3e5	BERT	3e-05	7.67 $\pm$ 1.15	98.87 $\pm$ 0.06	95.40 $\pm$ 0.28	97.57 $\pm$ 0.06	109.59M

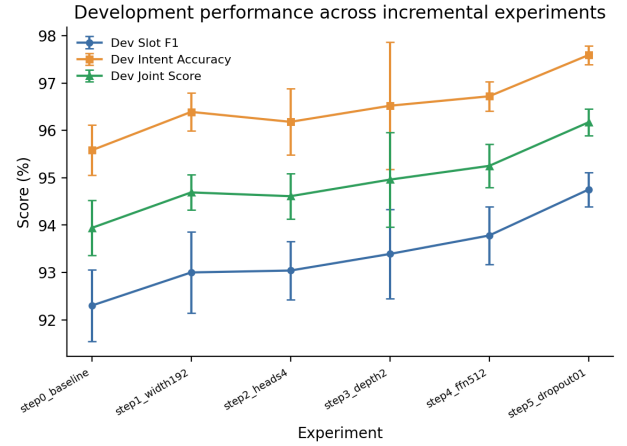
Pretrained models: openai-community/gpt2 and google-bert/bert-base-uncased.

Table 5: *Final test comparison between the selected scratch and pretrained models. Bold marks the stronger system.*

Part	Model family	Configuration	Slot F1	Intent Acc.	Joint
Part 2.A	Scratch Transformer	step5_dropout01	90.39 $\pm$ 0.48	92.83 $\pm$ 0.85	91.61 $\pm$ 0.36
Part 2.B	Pretrained BERT	bert_lr5e5	95.61 $\pm$ 0.07	97.69 $\pm$ 0.13	96.65 $\pm$ 0.08
Absolute gain	—	—	+5.22	+4.86	+5.04

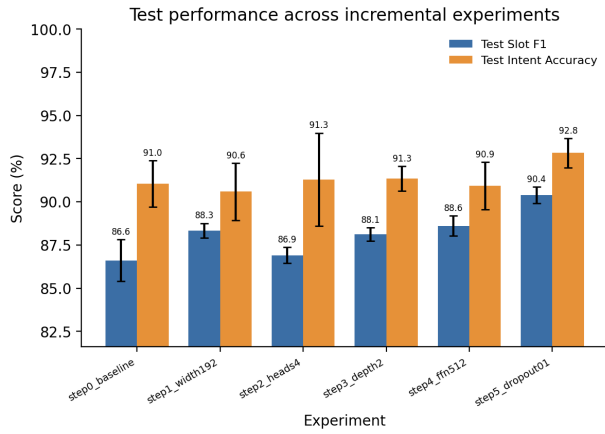


(a) Learning-rate search on mean development joint score.

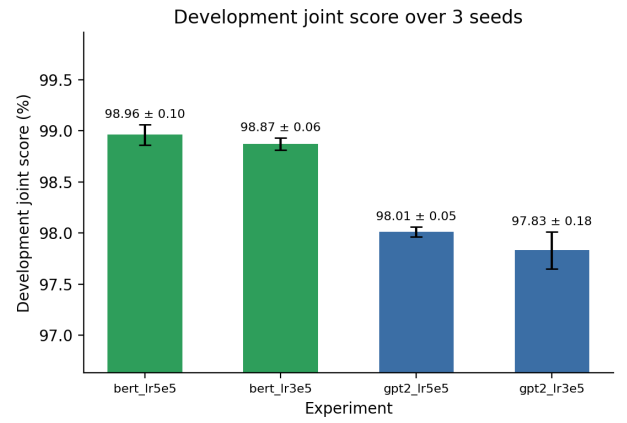


(b) Development metrics across the incremental changes.

Figure 1: Part 2.A development-stage model selection.

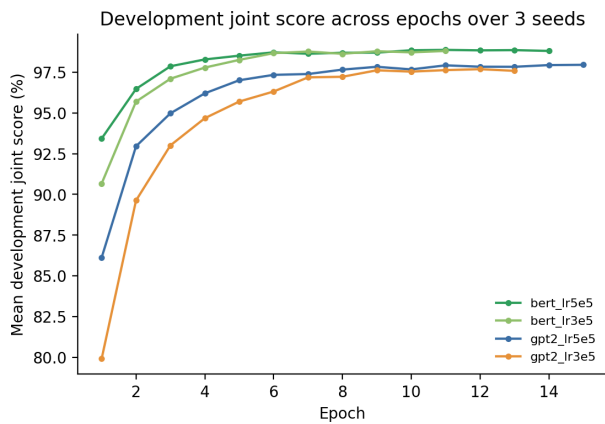


(a) Held-out test metrics for the Part 2.A scratch models.

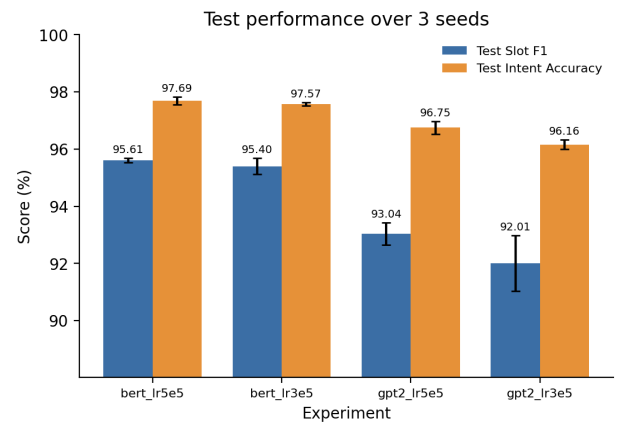


(b) Development joint-score comparison for the pretrained models.

Figure 2: Scratch-model test behaviour and pretrained-model development comparison.



(a) Part 2.B development joint score across epochs.



(b) Final held-out test performance of the pretrained models.

Figure 3: Part 2.B fine-tuning behaviour and final test performance.